Activation-Weighted Low-Rank Compression: Likelihood, Drift, and Practical Cost

Sambhav Surana

***Abstract*—Large language models use large matrices, or tables of numbers, to turn text into predictions. Replacing some of these matrices with two smaller ones can save storage, but the replacements may change how information passes through the model. We study whether a compression method guided by the model's observed inputs can preserve its predictions and the internal signals passed between layers. We compare this method with a version that also penalizes changes to the original weights, using the same number of stored values. Tests on three pretrained language models show that the penalty provides no consistent improvement in both prediction quality and internal similarity. In one model, prediction quality improves while average error in these internal signals grows to 3.86 times that of the method without the penalty. Tests on consecutive layers also find a group where both prediction quality and internal similarity worsen for all three choices of fitting data. Separate storage and timing tests remove 29.17% of the weights in the selected matrices, but reduce the complete model file by only 0.894%; execution time varies with the workload. These findings show that prediction quality alone cannot establish whether compression preserves the original computation. Failed full-rank SVD numerical controls, incomplete additional tests for one model, and evaluation on a previously used text dataset make the findings exploratory. A separate 270M-parameter GPU execution study preserves the compressed model's likelihood but establishes no acceleration with the tested kernels or per-projection graphs. A practical compression claim therefore needs separate evidence for prediction quality, internal behavior, storage, and speed.**

***Index Terms*—low-rank compression; activation weighting; representation drift; transformer evaluation; inference cost.**

I. Introduction

Language models use weight matrices to transform text into predictions. Low-rank compression replaces a large matrix with two smaller factors. The replacement saves weights, but its effect depends on the rest of the model. A factorized layer may closely match the original on calibration inputs and still receive different inputs after earlier layers have been compressed. A compressed model may also predict text more accurately while its internal representations move farther from those of the original model, which we call the teacher. We measure these outcomes separately.

We test whether a weight-space anchor improves prediction quality and internal similarity when the number of stored factor weights is fixed. Activation weighting gives more importance to input directions observed during calibration. The anchor adds a penalty for changing the original weights. It changes how the factors are fitted but adds no branch at inference. We compare it with covariance weighting without the anchor, using the same factor budget, and measure storage and execution cost. In the charts, "Ours (anchored)" identifies this tested anchored configuration.

The study separates the effects of activation weighting, anchoring and extra ridge shrinkage. We test attention-output replacements across the full stack and in consecutive groups, measuring prediction quality alongside the changes that propagate through the residual stream. We also measure complete checkpoint size and runtime. The results show where an average joint improvement fails to hold across calibration seeds or layer groups, and where better prediction quality accompanies greater internal drift.

The results remain exploratory. Some full-rank SVD numerical controls failed, and Qwen's primary comparison is available without a complete validation rank frontier. The later practical cohort uses bfloat16 (BF16), has only two calibration seeds, and adds no BF16 drift measurements. We do not claim a new whitening algorithm or reliable preservation of the complete model's computation.

II. Related work and comparison scope

Data-aware low-rank approximation predates this study. DRONE and weighted language-model factorization establish earlier approaches that use data or task information when selecting a low-rank representation [1] [2]. ASVD combines activation-magnitude scaling with sensitivity-based rank selection; its v5 covariance variant, ASVD+, uses a Cholesky factor of the activation Gram matrix [3]. SVD-LLM v5 combines covariance whitening with sequential factor-wise low-rank adaptation fine-tuning, while SVD-LLM(W) denotes its whitening-only component [4]. Our controls use fixed ranks and omit rank search and recovery fine-tuning. They isolate the diagonal-scaling and covariance-weighting components of those methods. For invertible covariance in exact arithmetic, unanchored covariance weighting shares the whitening geometry; numerical cutoffs and regularization still require separate qualification.

Accumulated error and rank allocation motivate distinct comparisons. SVD-LLM V2 studies truncation and rank allocation [5]. The SAES-SVD and LACE-SVD preprints address accumulated/local error and cumulative correction, respectively [7] [8]. Zero Sum SVD considers loss-sensitive global component selection, and IO-SVD introduces input/output whitening with heterogeneous ranks [9] [10]. These methods remain candidates for future comparisons at matched budgets; we did not run them here. Related work on quantization-error propagation reinforces the need to evaluate downstream

effects, without making quantization and low-rank truncation equivalent [11].

Compression also depends on the deployed representation. GPTQ, AWQ and SparseGPT study quantization or pruning rather than our fixed-rank factors [12] [13] [14]. EoRA uses eigenspace-informed low-rank compensation [6]. SVDQuant illustrates the execution cost of separate branches in diffusion models, a different application domain [15]. These methods provide context for the practical measurements; we did not benchmark them or establish faster language-model inference. Centered kernel alignment and temperature scaling provide representation and calibration diagnostics, respectively; neither establishes teacher preservation [16] [17].

III. Methods

A. Representation and parameter budget

For a projection $W \in \mathbb{R}^{o\times i}$, replace $Wx+b$ with $U(Vx)+b$, where $U \in \mathbb{R}^{o\times r}$ and $V \in \mathbb{R}^{r\times i}$. The original bias is applied once. The factors contain $r(o+i)$ weights, compared with oi in the dense matrix. Parameter savings therefore require

$$r < \frac{oi}{o+i}. \tag{1}$$

Rank reduction by itself does not specify the storage saving.

Only the output projections of full-attention blocks are replaced. Attention heads are not pruned. Embeddings, feed-forward layers and other untargeted weights remain dense; Qwen's DeltaNet and LFM's convolutional mixers also remain dense. For Gemma, each of 26 targets has shape 1152 by 1024. Rank 384 retains 835,584 of 1,179,648 weights per target: a 29.17% targeted reduction, or 8,945,664 weights across the stack. This fraction is not the whole-model saving.

The hybrid primary ranks follow

$$r = \left\lfloor \frac{17}{24}\,\frac{oi}{o+i} \right\rfloor, \tag{2}$$

giving rank 483 for Qwen and 725 for LFM. This approximately matches Gemma's retained target-weight fraction, rather than imposing the same integer rank on different shapes. Validation-only frontier ranks use

$$\left\lfloor \frac{\min(o,i)}{d} \right\rfloor, \tag{3}$$

with $d \in \{2,3,4\}$. Within each comparison, the anchor and whitening methods use identical target shapes, ranks and factor precision.

B. Weighting, anchoring and extra ridge

Let $X \in \mathbb{R}^{i\times n}$ contain calibration activations and write $A = UV$. The anchored objective is

$$J_\beta(A) = \|(W-A)X\|_F^2 + \beta n\|W-A\|_F^2, \tag{4}$$

subject to rank at most r. The first term weights directions by observed activation energy; the second penalizes weight-space deviation even in weakly observed directions. The anchor changes the fitting geometry while keeping the factor budget and inference graph fixed.

With exact teacher outputs, define $H = XX^T + \beta nI$ and $C = WH$. The BF16 practical implementation instead fits measured outputs: it first removes the bias, $Y_b = Y - b\mathbf{1}^T$, and minimizes

$$\|Y_b - AX\|_F^2 + \beta n\|W-A\|_F^2, \tag{5}$$

giving $C = Y_bX^T + \beta nW$. This equals the exact-output objective only when $Y_b = WX$; finite-precision measured outputs need not satisfy that identity. The historical extra-ridge solver uses $G = H + \lambda I$, with $\lambda = \rho\,\mathrm{mean}(\mathrm{eig}(H))$. It forms $A_{\mathrm{full}} = CG^\dagger$, truncates $A_{\mathrm{full}}G^{1/2}$ to rank r, and maps back using $G^{\dagger/2}$.

For nonsingular H, exact outputs and no extra ridge, the solution is

$$A_\beta = [WH^{1/2}]_r H^{-1/2}, \tag{6}$$

where brackets denote the best rank-r singular-value truncation. Without rank reduction, this recovers W. For singular or numerically truncated geometry, recovery is restricted to retained spectral directions. In contrast, positive extra ridge generally shrinks the unrestricted solution even before truncation. Consequently, an effect of the historical solver cannot automatically be attributed to low rank or to anchoring alone.

The unanchored whitening component sets $\beta = \rho = 0$. The selected anchor uses $\beta = 0.03$ and $\rho = 0$. Plain singular value decomposition (SVD) is a weight-space control. The fixed-rank diagonal ASVD component uses $s_j = \max(\mathrm{mean}_t\,|X_{jt}|, 10^{-8})^{0.5}$, truncates $W\,\mathrm{diag}(s)$ and scales back. Covariance fitting and weighted decomposition use 64-bit floating-point computation (FP64), then 32-bit floating-point factors (FP32) before the declared storage cast; plain SVD uses FP32. The weighted implementation retains regularized eigenvalues strictly above $10^{-7}\lambda_{\max}(G)$ and clamps negative numerical eigenvalues to zero. These finite-precision choices limit exact-arithmetic equivalence claims.

C. Local error and propagated residual drift

Local error compares the original and replacement projection on identical teacher activations. Propagated drift instead compares the outputs of every complete block in independent teacher and compressed-model forward passes on the same token sequence. The student therefore receives its own upstream hidden states. Both models use the observed token context; this is not free-running generated-text evaluation.

Let $h_{\ell,t}$ and $\widehat{h}_{\ell,t}$ be teacher and student block-output states at scored positions $t \in \mathcal{T}$. We use

$$D_\ell = \frac{\sum_{t\in\mathcal{T}} \|\widehat{h}_{\ell,t} - h_{\ell,t}\|_2^2}{\max\left(\sum_{t\in\mathcal{T}} \|h_{\ell,t}\|_2^2,\, 10^{-30}\right)} \tag{7}$$

and

$$D = \frac{1}{L}\sum_{\ell=1}^{L} D_\ell \tag{8}$$

over all model blocks. Errors and reference energy accumulate across all scored windows before normalization. The metric D averages the energy ratio for each block. It differs from both a last-layer error and an average of per-token ratios. For subgroup replacements, it includes unchanged upstream blocks as well as downstream blocks, so group location affects its interpretation.

Cosine distance, residual norm ratios, window-based linear centered kernel alignment (CKA), and teacher-to-student logit Kullback-Leibler (KL) divergence are secondary diagnostics. Low drift indicates similarity in the measured representation coordinates; it is not a theorem of functional equivalence. A representation change can preserve or improve an output metric. Conversely, a favorable average drift score need not guarantee good final logits or likelihood.

IV. Experimental protocol and evidence boundaries

A. Data, selection and uncertainty

All reported quality comparisons use the native WikiText-2 splits [19]. Calibration draws 16 disjoint chunks of 512 tokens from training data, paired across methods for seeds 11, 23 and 37 in the historical quality cohorts. Native validation selects regularization; native test evaluates the frozen setting. Rows retain dataset order and are joined with two newlines without added special tokens. Evaluation uses context 512, stride 256, exactly-once shifted-target masking and the final partial window. Negative log-likelihood (NLL) is total token loss divided by scored targets; perplexity (PPL) is its exponential. Native-tokenizer metrics are compared within models, never used to rank different tokenizers.

The original anchor sweep is $\beta \in \{0, 0.03, 0.1, 0.3, 1\}$. Extra-ridge fractions are $\rho \in \{0, 0.001, 0.01\}$ at $\beta = 0.1$. Validation NLL ties within 10^{-6} prefer less regularization. The selected ridge-free anchor is frozen before the primary comparisons; hybrid validation frontiers are development evidence, not additional confirmation.

Tables report calibration-seed means and sample standard deviations, with seed-independent SVD fitted once. Seeds describe calibration variation, not independently pretrained models. Paired 95% bootstrap intervals use 2,000 resamples of contiguous 4,096-target evaluation blocks. The incomplete tail remains in full-test NLL but is excluded from fixed-block resampling. These intervals therefore describe evaluation-block variation conditional on the fitted model; it does not replace seed uncertainty. Comparisons across many groups and ranks remain descriptive rather than multiplicity-corrected discoveries [18].

B. Numerical controls, coverage and practical amendment

The original strict numerical check for full-rank SVD factorization in 16-bit floating-point (FP16) failed. Copied-teacher and factor-split identity controls can match exactly, but do not establish that SVD-based factor preparation has equivalent numerical behavior. Later BF16 controls likewise do not retroactively validate the FP16 cohort: the BF16 full-rank SVD control has NLL 3.452658 versus 3.451550 for its teacher and fails its recorded probe and NLL gates. In the later BF16 cohort, higher-precision execution improves NLL agreement while the strict probe still fails; separate FP16 diagnosis variants passed both recorded gates. These failures concern the tested full-rank SVD controls and do not establish that the whitening or anchored fits failed the same checks. Conversely, those fits also use weighted SVD during factor preparation, so the SVD-control results alone do not establish their numerical equivalence. Matched comparisons remain observations of the recorded implementations, rather than isolated effects of ideal mathematical objectives.

The Qwen continuation contains all eight primary-test records but only 10 of 18 declared validation-frontier records. The continuation reuses observations from an interrupted attempt; they do not count as independent replications. LFM's eight primary-test and 18 frontier records are complete. Completion of either dispatcher or saved rows does not remove the readiness limitation.

A later practical amendment evaluates only BF16 Gemma: one dense teacher and whitening/anchor factors for seeds 11 and 23, five conditions in total. It is separate from the stopped FP16/INT8 practical cohort and does not supply new drift evidence. Its two seeds cannot satisfy the original three-seed promotion gate. No missing frontier point, seed, corpus or task is inferred from execution completion.

C. Separate 270M GPU execution cohort

An execution-only follow-up uses pretrained Gemma 3-270M in FP32 on an NVIDIA GeForce GTX 1050 (4 GB), with PyTorch 2.7.1+cu118 and Transformers 4.55.4. It reuses one saved whitening fit (seed 11), replacing all 18 attention-output projections, each $1024 \to 640$, with rank-275 factors. No activation, refitting or anchor is added. The unchanged factors are executed with two PyTorch linear operations, two custom CUDA stages that compute the intermediate once, or per-projection CUDA Graphs of the custom stages or cuBLAS operations. Kernels use NVRTC 11.8 [25]; graphs use PyTorch's capture/replay interface [26]. All new paths retain GPU cuBLAS for multi-token prefill.

This cohort uses its own pinned token manifests: 16 training and eight validation windows of 256 tokens. Native WikiText-2 test evaluation uses context/stride 256/128, includes the final tail, and scores each of 292,281 shifted targets once. A separate teacher-forced diagnostic scores test positions 128–255 after a 128-token prompt to exercise the single-token execution paths. It is not a full-test estimate. Batch-one GPU timings use five warmups, 20 synchronized samples per condition and rotating method order. Prefill is measured at 128/256 tokens; the other workload includes a 128-token prompt plus 32 cached steps, rather than decode alone. Initialization and capture are excluded from warmed timings. This cohort is not pooled with the earlier FP16/BF16 or three-model quality comparisons.

V. Results

A. Activation-aware fitting improves on plain SVD, but anchoring has a small mixed Gemma effect

Fig. 1 compares all 26 Gemma attention-output projections at rank 384 using the historical FP16 cohort. The dense reference NLL is 3.451563. Activation-aware components have substantially lower NLL and drift than plain SVD at the same factor budget. The anchor's mean NLL is slightly lower than whitening's, but both remain worse than the dense reference on this corpus.

Anchoring changes mean NLL by -0.006893 relative to whitening. The mean drift also decreases slightly, from 0.010638 to 0.010547, but drift worsens in two of the three paired seeds. Therefore the aggregate does not establish a reproducible joint improvement. The historical extra-ridge setting is worse than whitening on both metrics in every paired seed. The validation full-rank ridge control further demonstrates why shrinkage must be distinguished from rank reduction; an unrestricted-rank control is not a matched compressed baseline.

B. A full-stack average does not predict every consecutive group

The completed composition cohort evaluates single projections, groups of two, groups of four and all 26 projections. Smaller groups occur at early, middle and late positions, with fixed per-target rank. Fig. 3 reports paired mean differences, anchor minus whitening; negative values favor the anchor. It uses only the unique completed cohort. Interrupted predecessors remain in the evidence ledger and are not counted as new observations.

For middle blocks 11-14, anchor NLL is 3.475825 versus 3.473102 for whitening, and drift is 0.00044914 versus 0.00038059. Both metrics worsen in all three seeds. Other groups have mixed signs or much smaller differences. The full-stack average gain therefore does not hold for every group. Drift also need not grow monotonically with group size: layer position, downstream sensitivity and the unchanged blocks included in the average all affect the result.

C. Hybrid-model results separate likelihood from teacher-state fidelity

Fig. 4 shows primary-test means for the two hybrid models. Each method targets six full-attention output projections, with matching ranks within model. Qwen's rank is 483 and LFM's is 725. Their other mixer families remain dense. Three calibration seeds are present for each activation-aware method; the dense teacher and plain-SVD controls are recorded once.

Qwen's anchor has higher NLL and higher drift than whitening in every seed. This weakens the anchor hypothesis at its measured primary rank, even though its incomplete validation frontier prevents a complete frontier claim.

LFM shows the clearest disagreement between the metrics: anchor NLL is lower in every seed, while drift increases in every seed and is approximately 3.86 times the whitening mean. Its anchor NLL is also below the dense teacher's 3.998736, whereas its drift remains nonzero and much larger than whitening's. Better observed next-token likelihood therefore does not certify reproduction of the teacher's hidden-state computation. The result alone also cannot identify denoising, beneficial regularization or a particular causal mechanism. The three fully measured LFM validation ranks show the same qualitative likelihood/drift tradeoff, but remain development observations.

TABLE I
SEPARATE BF16 PRACTICAL COHORT. TEMPERATURE IS SELECTED ON VALIDATION FROM A BOUNDED GRID, THEN FROZEN; ALL FIVE SAVED SELECTIONS ARE APPROXIMATELY 1.090508. GIB USES 2^{30} BYTES. THESE ARE DESCRIPTIVE FULL-TEST AGGREGATES WITHOUT PAIRED BLOCK INTERVALS.

BF16 condition	Seed	Raw NLL	Scaled NLL	Whole GiB
Dense teacher	-	3.451549	3.422601	1.862537
Whitening	11	3.556086	3.541726	1.845885
Anchor	11	3.556826	3.542255	1.845885
Whitening	23	3.568431	3.553267	1.845885
Anchor	23	3.562949	3.545341	1.845885

D. Targeted compression yields modest whole-model storage savings and mixed timing

The BF16 practical cohort's dense checkpoint contains 1,999,883,711 bytes; each factorized checkpoint contains 1,982,003,507 bytes, including archive overhead. The reduction is 17,880,204 bytes, or 0.894%. Identical anchor and whitening factor budgets yield identical checkpoint sizes. The 29.17% reduction in targeted weights therefore translates to less than 1% of this whole-model serialization, because most weights remain untouched.

Table I retains the individual BF16 conditions.

Temperature scaling reduces NLL for both the teacher and compressed conditions, without closing the teacher-relative NLL gap. This supports treating calibration as a control rather than interpreting raw likelihood differences as denoising. The practical BF16 rows are not pooled with historical FP16 quality or drift measurements.

Timing uses a resident NVIDIA GeForce RTX 4070 Ti, batch size one, 10 warmups and 30 synchronized samples. Prefill is measured at 128 and 512 tokens. Cached 128-token decode is timed separately and excludes the initial prompt pass. Table II compares anchor with same-seed BF16 whitening under identical workloads. Ratios below one indicate lower median latency; they are descriptive workload comparisons, not a new acceptance gate.

Median ratios vary by seed and workload: the anchor is slower for seed 11 at the 128-token conditions, while the observed seed-23 ratios favor it. Peak allocated/reserved memory and serialized bytes match between the paired factorized methods. These observations establish neither a consistent anchoring speed advantage nor general acceleration over the

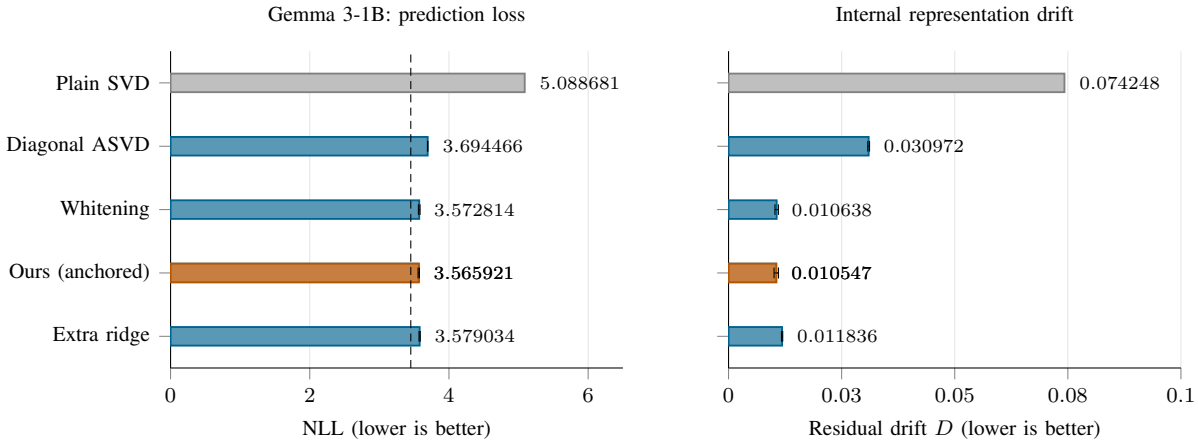
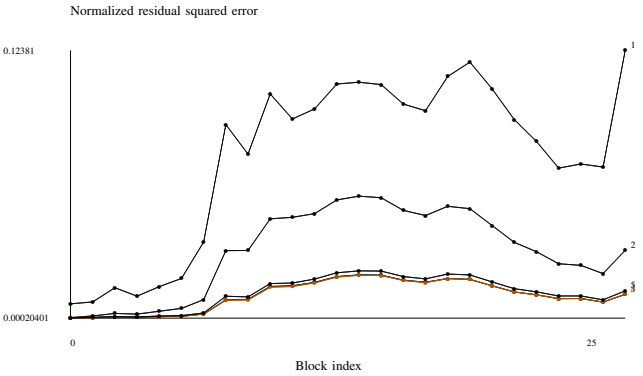


Fig. 1. Gemma full-stack native-test comparison at the same factor budget. Horizontal bars start at zero; orange identifies Ours (anchored). Bars show means and whiskers show sample standard deviation across three calibration seeds. Plain SVD is one seed-independent fit with no error bar. The dashed line is the dense teacher's NLL. Diagonal ASVD denotes the tested component; the anchor uses $\beta = 0.03$, and extra ridge uses $\beta = 0.1$, $\rho = 0.01$. Error bars are descriptive seed variation, not confidence intervals. The strict historical FP16 readiness failure still applies.



Series 1: SVD; 26 measured points.
Series 2: ASVD component; 26 measured points.
Series 3: Whitening; 26 measured points.
Series 4: Ours (anchored); 26 measured points.
Series 5: Extra ridge; 26 measured points.

Fig. 2. Gemma residual drift from the same saved coordinates. Individual-seed variation is retained in the companion PDF figure and original evidence.

TABLE II
MEDIAN LATENCY RATIOS, ANCHOR DIVIDED BY MATCHED WHITENING, FOR THE BF16 PRACTICAL COHORT. RATIOS BELOW ONE FAVOR THE ANCHOR.

Workload	Seed 11 median ratio	Seed 23 median ratio
Prefill 128	1.1876	0.9370
Decode 128 after prompt 128	1.1073	0.9679
Prefill 512	1.0108	0.9473
Decode 128 after prompt 512	0.9876	0.9580

dense teacher. CUDA allocator peaks are not total device memory or system RAM. Whole-model reload probes cover a 128-token prompt and one cached next-token forward, not arbitrary generation or the entire test corpus.

TABLE III
SEPARATE GEMMA 3-270M FP32 GPU EXECUTION STUDY. LOWER IS BETTER. LATENCIES ARE HOST-WALL MEDIANS IN MILLISECONDS, INCLUDING WRAPPERS, ALLOCATION, PROMPT TRANSFER AND CACHE HANDLING. "P+D" INCLUDES PROMPT 128 PLUS 32 CACHED STEPS. DENSE PPL IS REMEASURED, BUT DENSE TIMINGS ARE OMITTED BECAUSE THE ROTATING-ORDER TIMING COHORT DID NOT INCLUDE IT. EACH PPL USES ALL 292,281 TEST TARGETS. NEW EXECUTION METHODS SHARE GPU CUBLAS PREFILL.

Execution	PPL	Prefill 128	Prefill 256	P+D
Dense reference	47.2231	–	–	–
PyTorch factors	58.3059	46.273	66.184	1866.640
Two custom stages	58.3059	46.994	65.906	1921.132
Graph (custom)	58.3059	47.286	65.813	1940.790
Graph (cuBLAS)	58.3059	46.944	66.099	1939.413

E. Equivalent factor execution does not establish acceleration

Table III reports the separate 270M cohort. All factor execution paths have measured native-test PPL 58.305858, versus 47.223106 for the dense teacher. Full-sequence PPL uses their shared cuBLAS prefill path. The 128-target teacher-forced diagnostic exercises the new decode paths: its PPL is 58.921703–58.921705 for the factorized methods and 42.895796 for dense. These subset values must not be substituted for full-test PPL. Eight validation windows and 16 greedy positions also pass numerical agreement checks against the PyTorch factors, with matching checked token choices.

None of the tested custom or graph paths establishes a latency advantage over PyTorch. Graph timings include copying new inputs and cloning outputs to prevent replay from overwriting caller-held results. The prompt/decode PyTorch p95 is 2.489 s; new-path p95 values span 2.437–2.899 s, so small median differences are descriptive, not statistically established effects. The saved dense and whitening checkpoints occupy 1,072,489,021 and 1,058,261,499 bytes, respectively (1.327% reduction); the execution variants reuse the same

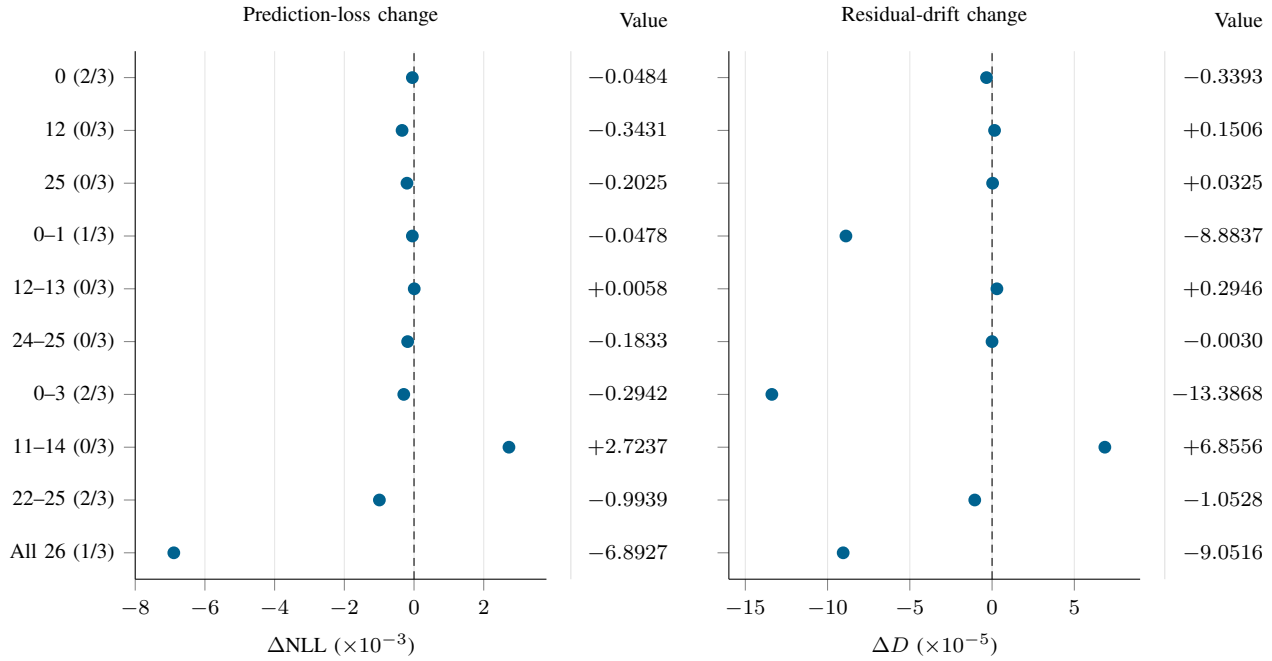


Fig. 3. Composition differences, Ours (anchored) minus whitening, across three calibration seeds, with zero-based block indices. Negative values favor the anchor over whitening; dashed lines mark no change. Axis units are 10^{-3} for NLL and 10^{-5} for drift, so the tick labels avoid long decimals. Values in the aligned side columns use these scaled units, with explicit signs and four decimal places. Each value shares its dot's row. Parentheses give the number of seeds, out of three, with strictly lower NLL and strictly lower D together. These are paired mean differences and descriptive counts, not significance tests.

factor parameters, while graph buffers add runtime state. Retained reference/graph buffers prevent interpreting allocator readings as isolated deployment-memory savings. These results concern the tested implementations and device, not custom GPU kernels in general.

VI. Discussion

LFM predicts text better under the anchored fit while its residual representations differ more from the teacher's. This result limits what likelihood alone can tell us about preservation. Residual similarity also has limits: D compares states in fixed coordinates, and different states can implement similar functions. A stronger preservation claim needs agreement across final logits, task performance, drift measurements and generation tests. Smaller storage or a better score on one measure does not supply that evidence.

Matched covariance weighting is the relevant baseline for the anchor. Gains over plain SVD show the value of data-aware fitting in this implementation, but cannot establish what the anchor adds or whether it is novel. Gemma's small average advantage, Qwen's joint worsening and LFM's trade-off provide no consistent basis for adding the same anchor across models. The models also differ in pretrained weights, tokenizers, mixers and historical inference precision, so we cannot attribute those outcomes solely to architecture.

The numerical controls constrain causal interpretation within their tested scope. Full-rank SVD failed the predefined tolerances in the recorded low-precision controls; this is not a demonstrated failure of the whitening or anchored fits. Method-specific full-rank checks are needed to establish numerical equivalence for those pipelines. Their matched comparisons remain informative, but factor casting, multiplication order, spectral cutoffs and finite-precision execution may contribute to measured differences. An exact identity split does not validate a different decomposition path.

Model-level storage and speed depend on which layers are compressed and how the factors execute. A targeted parameter saving can be diluted by embeddings and other dense components. Two dependent matrix multiplications can incur launch and intermediate-memory costs despite using fewer weights. Faster covariance preparation, fused BF16 factor execution and runtime-aware rank allocation are reasonable future hypotheses, but require new frozen protocols and matched quality/resource budgets. They are not measured benefits of the present study.

The 270M execution cohort distinguishes agreement with a compressed reference from agreement with the original teacher. Executing the same factors accurately preserves their approximation error; fewer launches or smaller intermediate storage cannot restore discarded model information. Its unchanged perplexity and absent measured speed advantage reinforce the need to validate quality and execution cost separately.

VII. Limitations and Reproducibility

WikiText-2 has already influenced this project; native-test results are regression evidence, not a fresh-domain confirmation. No downstream task or free-running generation

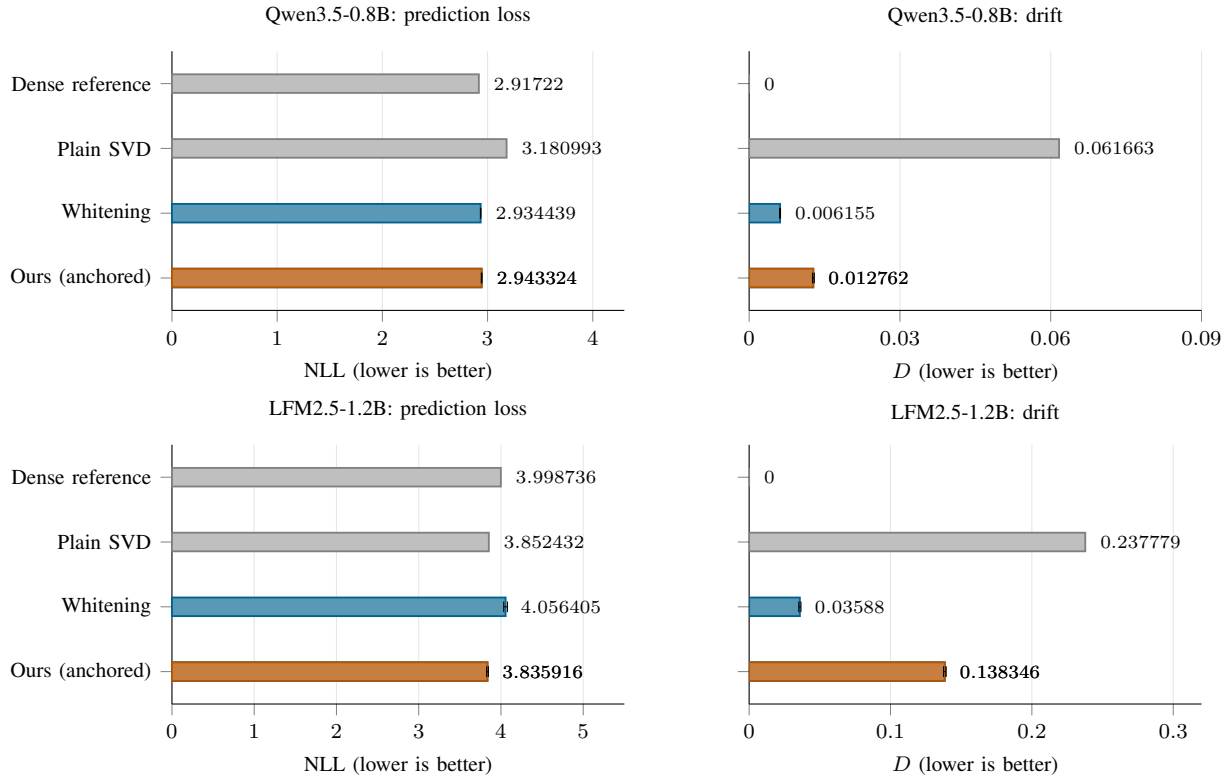


Fig. 4. Within-model hybrid primary-test comparisons. Horizontal bars start at zero; orange identifies Ours (anchored). Whitening and Ours show three-seed means and sample seed standard deviations. Dense reference and plain SVD are single recorded controls with no error bars; the dense model is an uncompressed reference. Factor ranks are 483 for Qwen and 725 for LFM. Model-specific axes show prediction loss and drift separately. Qwen coverage is 8/8 test; 10/18 validation. LFM coverage is 8/8 test; 18/18 validation. Cross-model differences do not isolate architecture effects. The anchor worsens both Qwen metrics, while LFM's lower NLL accompanies higher residual drift.

equivalence is established. A locked sample from C4, whose provenance is described in the T5 study, could provide fresh confirmation [20]; HellaSwag and ARC-Challenge could evaluate additional capabilities [21] [22]. None was executed here.

The scope is attention-output replacement at recorded budgets. Multilayer perceptrons (MLPs), hybrid non-attention mixers, larger models and other rank allocations require separate conclusions. Three calibration seeds provide limited variation, and the later BF16 practical amendment has only two. Within-model matching does not make different model families or FP16/BF16 cohorts interchangeable. The Qwen frontier's eight absent validation measurements remain missing. The failed full-rank SVD controls do not by themselves demonstrate a numerical failure of either weighted fit. Numerical equivalence for the whitening and anchored pipelines remains unestablished by those controls; together with incomplete coverage and unfulfilled promotion gates, this limits general preservation claims.

The tables and figures are reconstructed from the frozen evidence. The frozen evidence report (../../results/paper-v2/revised-report-20261014-151432-07c08d61/report.md) identifies the source runs, and the revision manifest records the exact extraction fields. Selected paired intervals remain in the repository's evidence file (report.json). The full historical report preserves commands, package/device versions, asset revisions, token manifests, factor recipes, source/checkpoint hashes, failed attempts and recovery accounting. Reused records are not independent replications. Recorded charged wall time includes failed/retried work and some reboot downtime; the first reboot has an unbounded unlogged tail, so no exact 24 GPU-hour compliance claim is made. The separate GPU follow-up is pinned to model revision 9b0cfec892e2bc2afd938c98eabe4e4a7b1e0ca1 and the recorded WikiText-2 revision/token hashes. It reuses one fitted checkpoint, not independent calibration repeats. Frozen records in results/gpu-lowrank-v2/20261016-mvb5e5qd and results/gpu-perplexity/20261016-mvb6ajxf retain contracts, source/checkpoint hashes, raw timing samples, complete target coverage and audits. Its 600-second execution study and 3,600-second PPL evaluation completed within their separate caps; startup failure evidence from the earlier kernel attempt is preserved. This editorial revision adds those saved records without rerunning evaluation or changing any earlier fit. Its one-device timing results and short decode diagnostic cannot establish general acceleration or free-running equivalence.

VIII. Future Scope

A. Compression with a small neural network

A possible extension is to replace a selected dense projection with a small bottleneck neural network. Its input would pass through a narrower hidden layer before returning to the original output dimension, allowing a compact mapping to be learned from the original model's activations. Transformer knowledge distillation provides a related precedent for training smaller students from larger teachers [24]. The proposed projection-level replacement would still require evaluation in the models and compression setting studied here.

Another option is to keep the low-rank factors and train a small network to correct their output error. The factor rank would need to leave enough room for this correction network within the matched storage budget. Both options should count all network weights and biases, together with any retained factors. A reduction in local error would need to improve held-out predictions and control residual drift when the replacements are composed across layers. Neither option has been implemented or tested in this study.

B. Numerical checks and broader evaluation

Further work should explain the failed full-rank SVD controls, establish method-specific full-rank numerical checks for whitening and anchoring, and complete the missing Qwen validation measurements. Controlled comparisons could then test whether matching the output of a group of layers reduces the downstream drift seen after composition. Fresh evaluation data, downstream tasks and generated-text tests would help assess whether any improvement extends beyond the text already used in this project. The model, data splits, fitting budget and decision criteria should be fixed before each comparison, with replication across calibration seeds.

C. Storage and execution cost

The small-network proposals also need measurements of complete checkpoint size, peak memory and warmed-up inference latency. Fused factor execution and rank allocation informed by runtime cost are further candidates for study. Each must be compared with matched low-rank baselines under the same workloads, since fewer stored weights alone do not establish a practical speed improvement.

IX. Conclusion

Activation-aware low-rank fitting improves on plain SVD at the tested attention-output budgets. Adding the anchor provides no consistent joint improvement in likelihood and residual fidelity over matched whitening. Gemma's small average gain does not hold across all layer groups. Qwen worsens on both metrics, while LFM improves likelihood and increases drift. The separate BF16 practical cohort saves 0.894% of the complete checkpoint size and has mixed timing results. The additional 270M FP32 GPU cohort preserves the low-rank reference's measured likelihood but establishes no execution speedup. Compression remains unproven as a way to preserve the original computation across the tested models.

Acknowledgment

OpenAI Codex [23] assisted with drafting and editing the Abstract, Introduction, Methods, Discussion, Future Scope and Conclusion, and with preparing formatting, table-assembly and plotting scripts and prototyping GPU execution code. Numerical results in this revision come from the saved experiment records; this editorial work did not run a new compression experiment.

References

[1] Patrick Chen, Hsiang-Fu Yu, Inderjit S. Dhillon et al.. DRONE: Data-aware Low-rank Compression for Large NLP Models. Advances in Neural Information Processing Systems, 2021. Primary source.

[2] Yen-Chang Hsu, Ting Hua, Sung-En Chang et al.. Language Model Compression with Weighted Low-Rank Factorization. International Conference on Learning Representations, 2022. Primary source.

[3] Zhihang Yuan, Yuzhang Shang, Yue Song et al.. ASVD: Activation-aware Singular Value Decomposition for Compressing Large Language Models. arXiv:2312.05821, 2023. Primary source.

[4] Xin Wang, Yu Zheng, Zhongwei Wan et al.. SVD-LLM: Truncation-aware Singular Value Decomposition for Large Language Model Compression. arXiv:2403.07378, 2024. Primary source.

[5] Xin Wang, Samiul Alam, Zhongwei Wan et al.. SVD-LLM V2: Optimizing Singular Value Truncation for Large Language Model Compression. Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025. Primary source.

[6] Shih-Yang Liu, Maksim Khadkevich, Nai Chit Fung et al.. EoRA: Fine-tuning-free Compensation for Compressed LLM with Eigenspace Low-Rank Approximation. arXiv:2410.21271, 2024. Primary source.

[7] Xing Hu, Dawei Yang, Yuan Cheng et al.. SAES-SVD: Self-Adaptive Suppression of Accumulated and Local Errors for SVD-based LLM Compression. arXiv:2602.03051, 2026. Primary source.

[8] Zhuowen Liu, Longkun Hao, Shiyu Feng et al.. LACE-SVD: Loss-Aware SVD with Cumulative Error Correction for LLM Compression. arXiv:2607.03057, 2026. Primary source.

[9] Ali Abbasi, Chayne Thrash, Haoran Qin et al.. Zero Sum SVD: Balancing Loss Sensitivity for Low Rank LLM Compression. arXiv:2602.02848, 2026. Primary source.

[10] Ali Abbasi, Chayne Thrash, Haoran Qin et al.. IO-SVD: Input-Output Whitened SVD for Adaptive-Rank LLM Compression. arXiv:2605.15626, 2026. Primary source.

[11] Yamato Arai, Yuma Ichikawa. Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization. arXiv:2504.09629, 2025. Primary source.

[12] Elias Frantar, Saleh Ashkboos, Torsten Hoefler et al.. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323, 2022. Primary source.

[13] Ji Lin, Jiaming Tang, Haotian Tang et al.. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978, 2023. Primary source.

[14] Elias Frantar, Dan Alistarh. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. arXiv:2301.00774, 2023. Primary source.

[15] Muyang Li, Yujun Lin, Zhekai Zhang et al.. SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models. arXiv:2411.05007, 2024. Primary source.

[16] Simon Kornblith, Mohammad Norouzi, Honglak Lee et al.. Similarity of Neural Network Representations Revisited. Proceedings of the 36th International Conference on Machine Learning, 2019. Primary source.

[17] Chuan Guo, Geoff Pleiss, Yu Sun et al.. On Calibration of Modern Neural Networks. Proceedings of the 34th International Conference on Machine Learning, 2017. Primary source.

[18] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika et al.. Lessons from the Trenches on Reproducible Evaluation of Language Models. arXiv:2405.14782, 2024. Primary source.

[19] Stephen Merity, Caiming Xiong, James Bradbury et al.. Pointer Sentinel Mixture Models. arXiv:1609.07843, 2016. Primary source.
[20] Colin Raffel, Noam Shazeer, Adam Roberts et al.. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. Journal of Machine Learning Research, 2020. Primary source.
[21] Rowan Zellers, Ari Holtzman, Yonatan Bisk et al.. HellaSwag: Can a Machine Really Finish Your Sentence?. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019. Primary source.
[22] Peter Clark, Isaac Cowhey, Oren Etzioni et al.. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457, 2018. Primary source.
[23] OpenAI. Codex. , 2026. Primary source.
[24] Jiao, Xiaoqi, Yin, Yichun, Shang, Lifeng et al.. TinyBERT: Distilling BERT for Natural Language Understanding. Findings of the Association for Computational Linguistics: EMNLP 2020, 2020. Primary source.
[25] NVIDIA. NVRTC (Runtime Compilation), CUDA Toolkit 11.8 documentation, 2022. Official documentation.
[26] PyTorch. CUDA semantics: CUDA Graphs, PyTorch 2.7 documentation. Accessed October 16, 2026. Official documentation.